

Triggers y funciones en SQL (PostgreSQL)

Diego Muñoz

1 Junio 2026

¿Qué es una función en SQL?

Bloque reutilizable de código

- Una **función** es un procedimiento que encapsula lógica en el servidor de base de datos.
- Puede recibir parámetros y retornar un valor o resultado.
- Se utiliza para cálculos, transformaciones, validaciones, etc.

Tabla principal: empleado

```
CREATE TABLE empleado (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT,  
    sueldo DECIMAL  
);
```

Tabla de auditoría: historial_sueldo

```
CREATE TABLE historial_sueldo (  
    id SERIAL PRIMARY KEY,  
    empleado_id INTEGER,  
    sueldo_anterior DECIMAL,  
    sueldo_nuevo DECIMAL,  
    fecha TIMESTAMP DEFAULT now()  
);
```

Tipos usados en el ejemplo

- SERIAL: entero autoincremental, sirve como llave primaria.
- TEXT: cadena de texto de largo variable.
- DECIMAL: número decimal exacto, sin errores de redondeo, apto para sueldos.
- INTEGER: número entero.
- TIMESTAMP: fecha y hora.

Datos de ejemplo

```
INSERT INTO empleado (nombre, sueldo) VALUES  
  ('Ana', 850000),  
  ('Bruno', 1200000),  
  ('Carla', 970000);
```

Usa `SELECT * FROM empleado;` para ver las filas cargadas.

Estructura de una función en SQL

```
CREATE FUNCTION nombre_funcion(parametro1 tipo, parametro2 tipo)
RETURNS tipo_retorno AS $$
BEGIN
    -- lógica
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;  -- lenguaje de la función
```

Funciones con parámetros y retorno

```
CREATE FUNCTION aplicar_aumento(  
    sueldo DECIMAL, porcentaje DECIMAL  
)  
RETURNS DECIMAL AS $$  
BEGIN  
    RETURN sueldo * (1 + porcentaje / 100);  
END;  
$$ LANGUAGE plpgsql;
```

- Se pueden usar directamente en consultas:

```
SELECT aplicar_aumento(sueldo, 10) FROM empleado;
```


Reglas y restricciones personalizadas

- Puedes usar triggers para validar condiciones más complejas que CHECK.
- Ejemplo: impedir dejar el sueldo en negativo.

```
IF NEW.sueldo < 0 THEN  
    RAISE EXCEPTION 'El sueldo no puede ser negativo';  
END IF;
```

CHECK con una función

Un CHECK puede llamar a una función para validar la fila. Primero la función que decide si un sueldo es válido:

```
CREATE FUNCTION sueldo_valido(s DECIMAL)
RETURNS BOOLEAN AS $$
BEGIN
    RETURN s > 0;
END;
$$ LANGUAGE plpgsql;
```

Tabla con CHECK

La tabla usa la función directamente en el CHECK:

```
CREATE TABLE empleado (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT,  
    sueldo DECIMAL CHECK (sueldo_valido(sueldo))  
);
```

Si el sueldo no cumple, el INSERT o UPDATE se rechaza.

Trigger para evitar cambios por usuarios no autorizados

```
IF current_user <> 'admin' THEN  
    RAISE EXCEPTION 'Solo el administrador puede modificar este campo';  
END IF;
```

- Ideal para auditoría o protección de datos críticos.

¿Qué es un trigger?

Un disparador automático

- Un **trigger** se ejecuta automáticamente cuando ocurre un evento: INSERT, UPDATE, o DELETE.
- Se asocia a una tabla específica.
- Ejecuta una función que define la lógica a realizar.

¿Por qué usar funciones y triggers?

- Para encapsular **reglas del negocio** directamente en la base de datos.
- Para implementar **auditorías**, validaciones o transformaciones automáticas.
- Para mantener **consistencia** incluso cuando acceden múltiples aplicaciones a la BD.

Eventos que pueden activar un trigger

- BEFORE INSERT, BEFORE UPDATE, BEFORE DELETE
- AFTER INSERT, AFTER UPDATE, AFTER DELETE

¿Por fila o por operación?

- FOR EACH ROW: se ejecuta una vez por cada fila afectada.
- FOR EACH STATEMENT: se ejecuta una vez por sentencia (poco común).

FOR EACH ROW vs FOR EACH STATEMENT

Tabla para registrar lo que pasa en cada UPDATE:

```
CREATE TABLE log_operacion (  
    id SERIAL PRIMARY KEY,  
    detalle TEXT  
);
```

Le conectamos dos triggers a empleado y comparamos qué deja cada uno.

FOR EACH ROW: la función

```
CREATE FUNCTION log_fila()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO log_operacion (detalle)  
    VALUES ('cambió empleado ' || NEW.id);  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

Usa NEW, tiene la fila de cada empleado cambiado.

FOR EACH ROW: el trigger

```
CREATE TRIGGER t_fila  
AFTER UPDATE ON empleado  
FOR EACH ROW  
EXECUTE FUNCTION log_fila();
```

Se ejecuta una vez por cada fila afectada.

FOR EACH STATEMENT: la función

```
CREATE FUNCTION log_sentencia()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO log_operacion (detalle)  
    VALUES ('UPDATE sobre empleado');  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

No usa NEW, registra la operación completa.

FOR EACH STATEMENT: el trigger

```
CREATE TRIGGER t_sentencia  
AFTER UPDATE ON empleado  
FOR EACH STATEMENT  
EXECUTE FUNCTION log_sentencia();
```

Se ejecuta una sola vez por sentencia.

Resultado

Un UPDATE que afecta a los 3 empleados:

```
UPDATE empleado SET sueldo = sueldo * 1.1;  
SELECT detalle FROM log_operacion;
```

cambió empleado 1

cambió empleado 2

cambió empleado 3

UPDATE sobre empleado

ROW dejó 3 filas, una por empleado; STATEMENT dejó 1, por el UPDATE.

Estructura de una función para trigger

```
CREATE FUNCTION nombre_trigger()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- lógica  
    RETURN NEW; -- o OLD  
END;  
$$ LANGUAGE plpgsql;
```

Objetos disponibles dentro de un trigger

- NEW: fila nueva (después de un INSERT o UPDATE).
- OLD: fila antigua (antes de un DELETE o UPDATE).
- TG_OP: operación que activó el trigger (INSERT, UPDATE, DELETE).
- TG_TABLE_NAME: nombre de la tabla que activó el trigger.
- TG_TABLE_SCHEMA: esquema de la tabla que activó el trigger.
- Y otros más.

Estructura de un trigger

```
CREATE TRIGGER nombre_trigger  
AFTER UPDATE ON tabla  
FOR EACH ROW  
EXECUTE FUNCTION nombre_funcion();
```


Función para registrar cambios

```
CREATE FUNCTION registrar_cambio_sueldo()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.sueldo <> OLD.sueldo THEN  
        INSERT INTO historial_sueldo (  
            empleado_id, sueldo_anterior, sueldo_nuevo  
        )  
        VALUES (OLD.id, OLD.sueldo, NEW.sueldo);  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Trigger asociado

```
CREATE TRIGGER trigger_sueldo_update  
AFTER UPDATE ON empleado  
FOR EACH ROW  
EXECUTE FUNCTION registrar_cambio_sueldo();
```

- Cada vez que se actualiza el sueldo de un empleado, se registra automáticamente en `historial_sueldo`.

```
UPDATE empleado SET sueldo = 1000000 WHERE id = 3;
```

Validación con BEFORE UPDATE

Un trigger BEFORE puede rechazar el cambio antes de guardarlo. Esta función impide dejar el sueldo en negativo.

```
CREATE OR REPLACE FUNCTION validar_sueldo()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.sueldo < 0 THEN  
        RAISE EXCEPTION 'El sueldo no puede ser negativo';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Trigger de validación

```
CREATE TRIGGER trigger_validar_sueldo  
BEFORE UPDATE ON empleado  
FOR EACH ROW  
EXECUTE FUNCTION validar_sueldo();
```

Si un UPDATE deja el sueldo bajo cero, la operación se cancela y ninguna fila cambia.

Una función resuelve los nombres de tabla con el `search_path` en tiempo de ejecución. Si tus tablas están en un esquema fuera de ese path, el trigger falla con `relation does not exist`.

Calificar el esquema dentro de la función:

```
INSERT INTO rrhh.historial_sueldo  
    (empleado_id, sueldo_anterior, sueldo_nuevo)  
VALUES (OLD.id, OLD.sueldo, NEW.sueldo);
```

Fijar el search_path de la función:

```
ALTER FUNCTION rrhh.registrar_cambio_sueldo()  
SET search_path = rrhh, public;
```


Una función para varios eventos

TG_OP indica qué operación disparó el trigger: INSERT, UPDATE o DELETE. Una sola función puede manejar las tres y decidir con un IF.

En un DELETE la fila está en OLD; en INSERT y UPDATE, en NEW.

Ejemplo con TG_OP

```
CREATE OR REPLACE FUNCTION auditar()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO log_operacion (detalle)  
    VALUES (TG_TABLE_NAME || ' ' || TG_OP);  
    IF TG_OP = 'DELETE' THEN  
        RETURN OLD;  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Disparar solo si cambia una columna

AFTER UPDATE OF sueldo ejecuta el trigger solo cuando esa columna aparece en el SET. Es más preciso que comparar NEW con OLD.

```
CREATE TRIGGER trigger_sueldo_update  
AFTER UPDATE OF sueldo ON empleado  
FOR EACH ROW  
EXECUTE FUNCTION registrar_cambio_sueldo();
```

Para cambiar la lógica usa `CREATE OR REPLACE FUNCTION`, sin borrar el trigger. Para quitarlos:

```
DROP TRIGGER IF EXISTS trigger_sueldo_update ON empleado;  
DROP FUNCTION IF EXISTS registrar_cambio_sueldo();
```

Dónde crear funciones y triggers

Funciones y triggers se crean con SQL directo en DBeaver o psql, donde el delimitador \$\$ funciona sin problemas.

En Antares SQL el editor corrompe el \$\$ por un bug abierto. Ahí debes usar su formulario visual o reemplazar \$\$ por comillas simples.

- Usar AFTER si se necesita trabajar con datos ya modificados.
- Siempre verificar si hay cambio antes de insertar auditoría.
- Evitar lógica compleja que duplique la del backend.
- Mantener las funciones versionadas y revisadas como cualquier código.

- Las funciones permiten encapsular lógica reutilizable en SQL.
- Los triggers ejecutan funciones automáticamente al cambiar datos.
- Se usan para auditorías, validaciones, transformaciones y protección.
- BEFORE valida o corrige antes de guardar, AFTER reacciona al cambio ya hecho.
- Las tablas que referencia una función deben estar en su `search_path`.

¿Tienes dudas? ¡Hablemos!